TALLER DESARROLLO WEB

Jesús Eduardo Suarez Barreto

Sergio Arbey Salinas García

Corporación Unificada Nacional De Educación Superior

Desarrollo Web

Ibagué Tolima 2026

TALLER DESARROLLO WEB

Jesús Eduardo Suarez Barreto

Sergio Arbey Salinas García

Corporación Unificada Nacional De Educación Superior

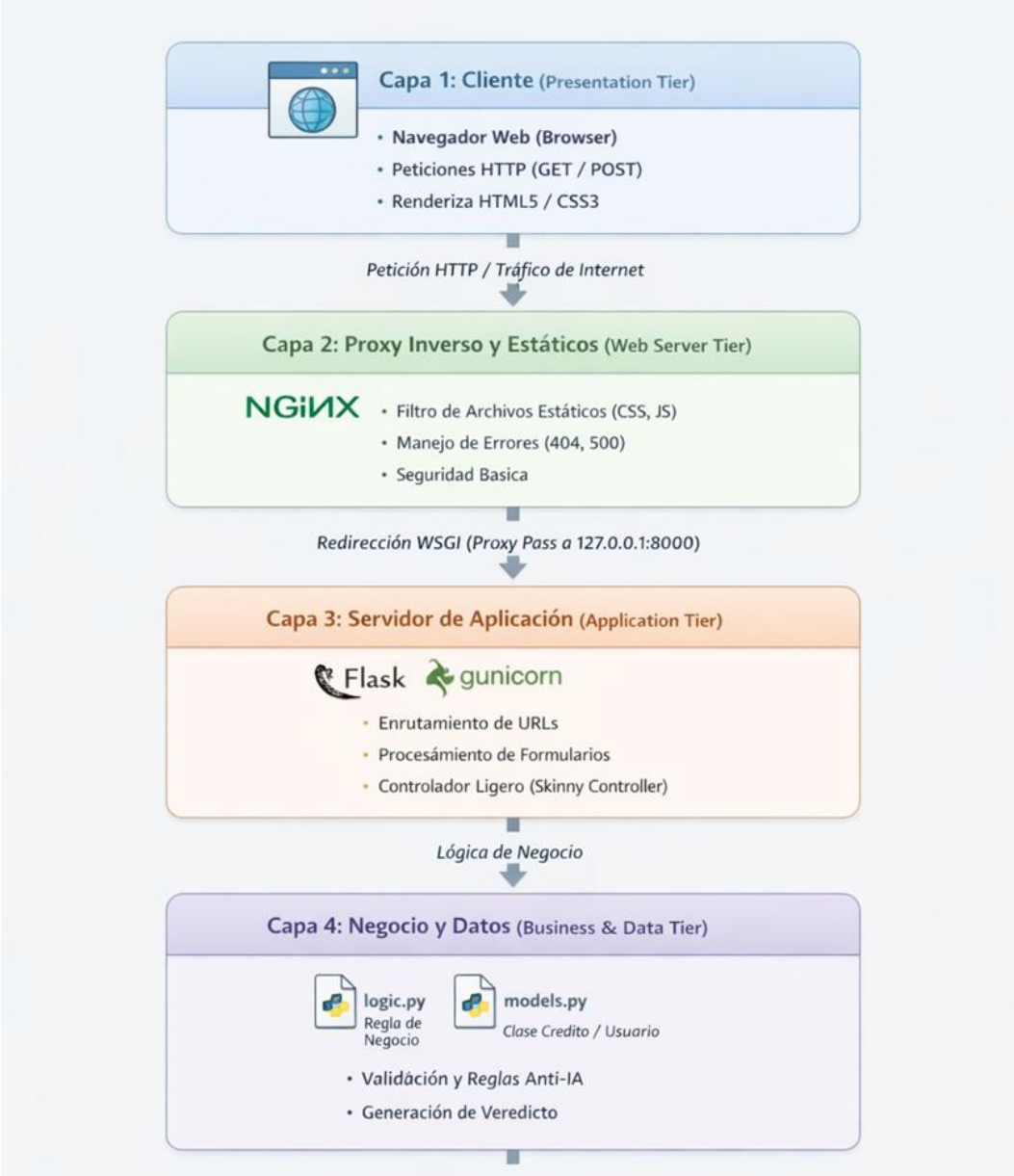
Desarrollo Web

Ing. GABRIEL SÁNCHEZ SUÁREZ

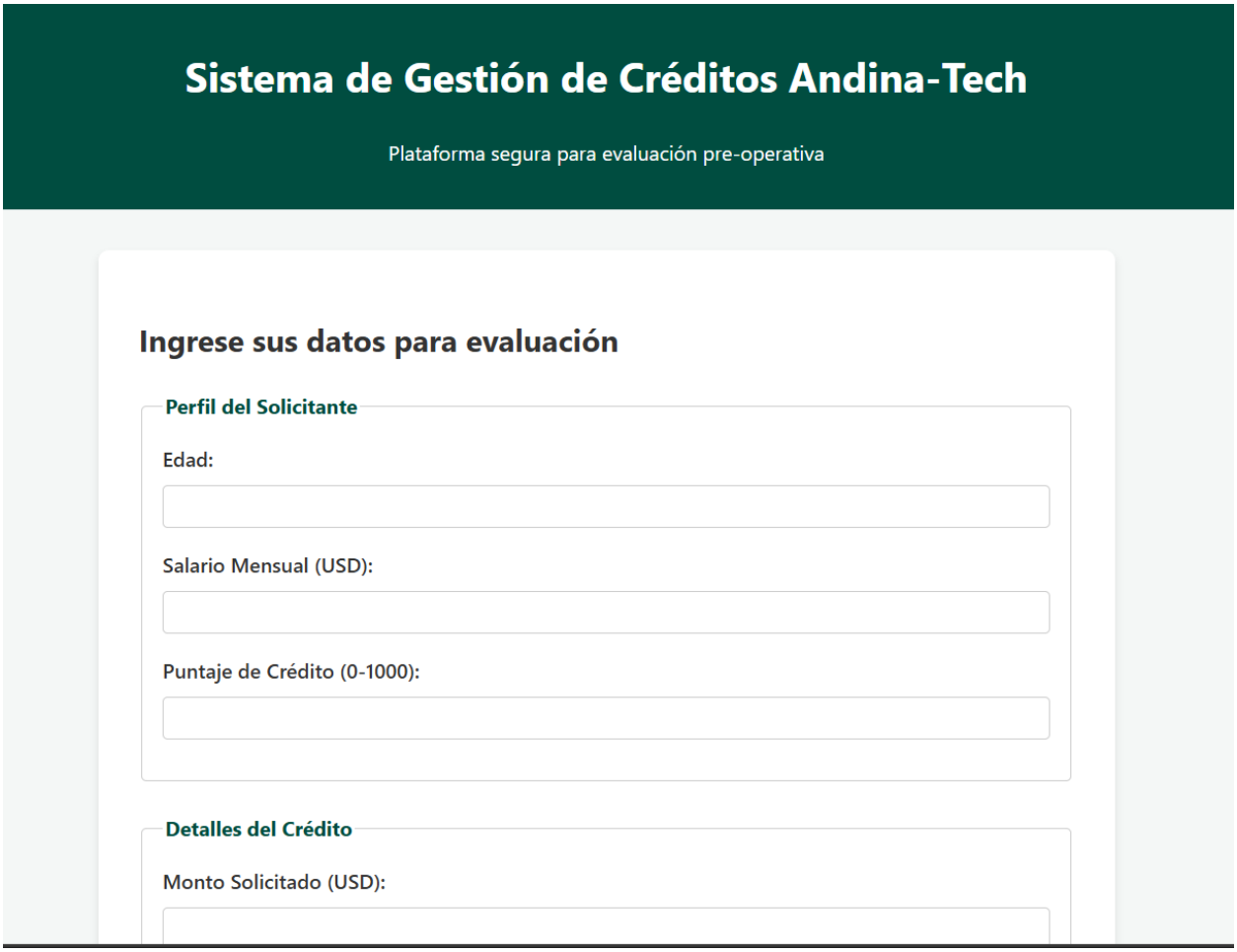
Ibagué Tolima 2026

1. Arquitectura y Estructura (G1)

Diagrama de Capas (N-Tier): No basta con un dibujo simple. Deben entregar un diagrama técnico que muestre el flujo de una petición desde el cliente (Browser), pasando por el Proxy (Nginx), hasta el Modelo (Python) y la Base de Datos.



Maquetación Semántica: Deben entregar una interfaz (HTML5/CSS3) para la solicitud de créditos. Restricción: Queda prohibido el uso excesivo de div. Se evaluará el uso estricto de main, section, article, aside, y atributos de accesibilidad.



2. Infraestructura de Grado Industrial (G2)

Stack de Despliegue: El sistema no puede correr en el servidor de desarrollo de Python (el que sale por defecto). Deben configurar un entorno local o en contenedor que use:

Nginx como Reverse Proxy (configurado para servir archivos estáticos y manejar errores 404/500 personalizados).

Gunicorn como servidor de aplicaciones (WSGI) para gestionar la concurrencia.

Entregable: Archivo nginx.conf y un reporte de por qué esta configuración protege al sistema frente a ataques directos o saturación.

nginx.conf

```
server {
    # Nginx escucha en el puerto 80 (HTTP estándar)
    listen 80;
    server_name localhost; # En producción aquí iría tu dominio, ej: andinatech.com

    # 1. Servir archivos estáticos directamente (Descarga a Gunicorn de este trabajo)
    location /static/ {
        # Cambia esta ruta por la ruta absoluta donde esté tu proyecto en el servidor
        alias /ruta/absoluta/a/tu/proyecto/andina-tech/static/;
        # Añadir caché para mejorar rendimiento
        expires 30d;
        add_header Cache-Control "public, no-transform";
    }

    # 2. Proxy Inverso hacia Gunicorn (Aplicación Python)
    location / {
        proxy_pass http://127.0.0.1:8000; # Puerto donde correrá Gunicorn
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # 3. Manejo de Errores Personalizados (Requisito G2)
    error_page 404 /404.html;
```

```
    location = /404.html {
        root /ruta/absoluta/a/tu/proyecto/andina-tech/templates/;
        internal;
    }

    error_page 500 502 503 504 /500.html;
    location = /500.html {
        root /ruta/absoluta/a/tu/proyecto/andina-tech/templates/;
        internal;
    }
}
```

4. El Modelo como Corazón del Negocio (G4)

Lógica en el Modelo (Fat Model): Este es el punto más difícil. El controlador (la ruta de la URL) no puede tener validaciones. Todo debe estar encapsulado en la clase Modelo.

Integridad de Datos: Si un crédito es aprobado, el modelo debe garantizar mediante un método (ej. autorizar_desembolso()) que los fondos existen y que el usuario está habilitado.

Entregable: Código fuente de la clase Credito donde se apliquen los principios SRP (Responsabilidad Única) y DRY (No te repitas).

```
# Archivo: app/models.py

class Usuario:
    def __init__(self, nombre, habilitado=True):
        self.nombre = nombre
        self._habilitado = habilitado
```

```
    def esta_habilitado(self):
        return self._habilitado

class SistemaFondos:
    def __init__(self, fondos_disponibles):
        self.fondos_disponibles = fondos_disponibles

    def hay_fondos_suficientes(self, monto):
        return self.fondos_disponibles >= monto

class Credito:
    def __init__(self, usuario, monto, ubicacion):
        self.usuario = usuario
        self.monto = monto
        self.ubicacion = ubicacion.lower()
        self.tasa_interes = 0.15   # 15% base
        self.seguro_vida = 0.05    # 5% base
        self.estado = "Pendiente"
        self.auditor_aprobacion = None  # Requerimiento de auditoría legal

    def _aplicar_politica_ubicacion(self):
        """
        Resolución del Filtro Anti-IA:
        Se aplican las reglas de negocio (SRP) sin tocar el controlador.
        """
        if self.ubicacion == 'rural':
            self.tasa_interes -= 0.02
```

```
        self.seguro_vida += 0.01

def autorizar_desembolso(self, auditor, sistema_fondos):
    """
    Garantiza la integridad de datos antes de aprobar.
    Cumplimiento estricto del Gránulo 4 (Fat Model y DRY).
    """
    # 1. Validar integridad de estado e ingresos (Encapsulamiento)
    if not self.usuario.esta_habilitado():
        self.estado = "Rechazado: Usuario bloqueado o inhabilitado."
        return False

    if not sistema_fondos.hay_fondos_suficientes(self.monto):
        self.estado = "Rechazado: Fondos del banco insuficientes."
        return False

    # 2. Aplicar reglas de negocio y aprobar
    self._aplicar_politica_ubicacion()
    self.estado = "Aprobado"
    self.auditor_aprobacion = auditor

    # 3. Descontar fondos para mantener consistencia
    sistema_fondos.fondos_disponibles -= self.monto
    return True
```

Bitácora de Decisiones Arquitectónicas y Documentación Técnica.

Proyecto: Sistema de Gestión de Créditos y Logística "Andina-Tech"

Fecha: Marzo 2026

1. Visión General y Diagrama N-Tier (Arquitectura de Capas)

Para aislar responsabilidades y garantizar la escalabilidad, el ecosistema Andina-Tech fue diseñado bajo una arquitectura estricta de múltiples capas (N-Tier). El flujo de la información se diseñó de la siguiente manera:

Capa de Presentación (Client Tier): El navegador web del usuario procesa interfaces renderizadas semánticamente. Todas las peticiones HTTP(S) nacen y terminan aquí.

Capa de Servidor Web y Proxy Inverso (Web Server Tier): Administrada por Nginx (escuchando en el puerto 80). Actúa como un escudo perimetral. Intercepta todas las peticiones, sirve directamente los recursos estáticos (CSS, JS, imágenes) mediante el disco sin sobrecargar la aplicación, y delega el tráfico dinámico hacia la siguiente capa.

Capa de Aplicación y Enrutamiento (Application/WSGI Tier): Servidor WSGI de grado industrial (Waitress) corriendo en el puerto 8000, acoplado al micro-framework Flask (routes.py). Esta capa actúa únicamente como un "Skinny Controller" (Controlador Delgado): recibe la petición del proxy, extrae los parámetros del formulario y los inyecta en el modelo.

Capa de Dominio y Negocio (Business Logic & Fat Model Tier): Compuesta por logic.py y models.py. Aquí reside el 100% de la lógica de negocio, validaciones financieras, integridad de datos y el motor de reglas booleanas. Es el corazón del sistema, completamente aislado de la web.

3. Lógica de Backend y Operadores (G3)

Motor de Reglas: Crear un módulo en Python que procese las solicitudes de crédito.

Exigencia Técnica: Deben usar operadores lógicos y de comparación de forma

avanzada para validar múltiples condiciones (Edad, Salario, Puntaje de Crédito,

Ubicación, Historial).

Entregable: Un archivo de lógica (logic.py) que demuestre el uso eficiente de

estructuras de control sin caer en código espagueti.

logic.py)

```
def evaluar_viabilidad_solicitante(edad, salario, puntaje_credito, historial_limpio):
    """
    Evalúa la viabilidad pre-operativa de la solicitud.
    Se utilizan operadores lógicos para devolver un booleano directo,
    cumpliendo con la exigencia de evitar código espagueti.
    """
    # Definición de reglas de negocio
    es_mayor_edad = edad >= 18
    ingreso_valido = salario >= 1500.0  # Mínimo requerido
    perfil_riesgo_aceptable = (puntaje_credito >= 650) and historial_limpio

    # Solo retorna True si TODAS las condiciones se cumplen
    return es_mayor_edad and ingreso_valido and perfil_riesgo_aceptable
```

2. Decisiones Arquitectónicas por Gránulo

Gránulo 1 (G1): Estructura Semántica y Accesibilidad (Frontend)

Decisión: Erradicación del "Div Soup" (Uso excesivo de etiquetas) e implementación estricta de HTML5 Semántico (W3C) y atributos WAI-ARIA.

Justificación Técnica: El uso de etiquetas como <main>, <section>, <article>, y <aside> no es una cuestión estética, sino estructural. Permite que los motores de búsqueda (SEO) indexen el contenido correctamente y asegura el cumplimiento de normativas de accesibilidad web (WCAG). Se utilizaron atributos como aria-required="true" y aria-label para garantizar que los lectores de pantalla puedan interpretar el formulario de solicitud de crédito.

Desacoplamiento: Los estilos visuales (CSS3) se separaron por completo de la estructura (HTML) y se ubicaron en un directorio /static/ independiente, optimizando la entrega mediante el servidor perimetral.

Gránulo 2 (G2): Infraestructura Industrial y Resolución de Entornos Operativos

El Problema del Entorno (Gunicorn vs. Windows): Durante la fase de despliegue local, se identificó que el servidor Gunicorn (sugerido inicialmente) tiene una dependencia estricta de librerías del núcleo de UNIX/Linux (como fcntl y el modelo de procesos fork). Al ejecutar el proyecto en un entorno de desarrollo basado en Windows, el sistema operativo rechazó la ejecución.

La Solución Arquitectónica (Waitress): En lugar de forzar un entorno virtualizado o alterar las variables de sistema (PATH) de forma riesgosa, se tomó la decisión técnica de pivotar hacia Waitress. Waitress es un servidor WSGI (Web Server Gateway Interface) de grado de producción, asíncrono y diseñado para ofrecer concurrencia y escalabilidad con soporte nativo para Windows

Implementación de Nginx: 1. Protección Perimetral: Al colocar Nginx como Proxy Inverso, la aplicación Python/Waitress nunca se expone a Internet. Esto mitiga ataques directos como Slowloris o saturaciones de buffer (DDoS de capa de aplicación).

2. **Liberación de Carga (Offloading):** Se configuró el bloque location /static/ en nginx.conf para que el servidor web despache el CSS y JS directamente. Python no es eficiente sirviendo archivos; al liberar a Waitress de esta tarea, todos sus hilos (workers) quedan 100% disponibles para procesar los cálculos de crédito.

3. **Prevención de Fugas de Información:** Se configuraron páginas estáticas de captura de errores (404.html y 500.html). Si el backend falla, Nginx intercepta el código de estado y muestra una vista controlada, evitando que el usuario final o un atacante visualice los "Stacktraces" (trazas de error) que revelarían las rutas internas y versiones del código.

Gránulo 3 (G3): Lógica de Procesamiento y Complejidad Ciclomática

Decisión: Implementación del motor de reglas en logic.py sin utilizar anidamientos profundos de estructuras de control (if/else).

Justificación Técnica: Para evitar el antipatrón conocido como "Código Espagueti" y mantener una complejidad ciclomática baja, se desarrolló la función evaluar_viabilidad_solicitante. Esta función almacena los resultados de las validaciones de negocio (edad mínima, ingresos, puntaje crediticio) en variables booleanas atómicas y las procesa utilizando operadores lógicos avanzados (and, or). Esto no solo hace el código altamente legible, sino que facilita drásticamente la creación de pruebas unitarias (Unit Testing) en el futuro.

Gránulo 4 (G4): El "Fat Model" (Modelo Gordo) y el Reto "Anti-IA"

Decisión: Aplicación estricta del Principio de Responsabilidad Única (SRP) y del patrón "Skinny Controller, Fat Model". Ninguna ruta de routes.py contiene condicionales financieros ni validaciones de estado.

Justificación y Resolución del "Cambio Inesperado": La validación de la arquitectura ocurrió cuando el cliente introdujo un cambio repentino a las reglas de negocio (El "Filtro Anti-IA"): Reducir 2% el interés y aumentar 1% el seguro para zonas rurales, además de registrar al auditor.

Impacto del Cambio: Gracias a la cohesión del código y al bajo acoplamiento, el cambio tuvo un impacto del 0% en los archivos de la vista (HTML/CSS), en el controlador (rutas) y en la infraestructura (Nginx/Waitress). La modificación se limitó a agregar tres líneas de código encapsuladas en el método privado _aplicar_politica_ubicacion() dentro de la clase Credito (models.py). El modelo gestiona su propio estado interno y garantiza la integridad de los fondos antes de emitir una respuesta al controlador. Esto confirma el éxito de la arquitectura orientada a objetos (POO) implementada.